

|                                                        |                                |
|--------------------------------------------------------|--------------------------------|
| <b>Materia:</b> Programación de Objetos Distribuidos   | <b>Código:</b> 72.42           |
| <b>Créditos:</b> 3                                     | <b>Carga horaria total:</b> 51 |
| <b>Departamento:</b> Sistemas Digitales y Datos        | <b>Año:</b> 2023               |
| <b>Carrera de:</b> Ingeniería en Informática           |                                |
| <b>Fecha última actualización:</b> 21/12/2022 12:46:47 |                                |

### ➤ Carga horaria:

| 51 | Horas totales      |
|----|--------------------|
| 19 | hs. teóricas       |
| 32 | hs. prácticas      |
| 0  | hs. de laboratorio |

| 3 | Horas semanales |
|---|-----------------|
| 3 | hs. presencial  |
| 0 | hs. a distancia |

### ➤ Contenidos mínimos:

Sistemas distribuidos: objetivos y tipos. Arquitecturas. Middleware. Threads. Virtualización. Clientes y Servidores. Migración de código. Invocación remota. Mensajes. Calidad de servicio. Diseminación de datos. Búsqueda. Sincronización. Consistencia. Replicación. Tolerancia a fallas. Recuperación. Seguridad.

### ➤ Presentación de la materia:

La intención de la materia es explorar las estrategias de programación concurrente y distribuida. Ambas estrategias permiten a su manera intentar obtener una mayor capacidad de cómputo y/o almacenamiento para que los sistemas puedan tener un mejor rendimiento. Estas estrategias traen aparejados nuevos potenciales problemas que deben ser prevenidos para evitar fallas o comportamientos no esperados.

Al final de la materia se espera que los alumnos puedan analizar y encontrar partes de los sistemas a construir donde estas estrategias puedan ser beneficiosas, implementar las soluciones y detectar posibles puntos de error inherentes a la estrategia utilizada.

La materia se dicta de manera presencial con una introducción teórico-práctica (a modo de taller) y prácticas realizadas por los alumnos.

### ➤ Objetivos de aprendizaje:

- Conocer nuevas funcionalidades y revisar funcionalidades conocidas de Java que puedan ser útiles en la materia como programación funcional, streaming y lambdas.
- Codificar sistemas concurrentes utilizando Java y las diferentes opciones para correr Threads en paralelo.
- Predecir e identificar problemas de concurrencia y fundamentar la elección de protección contra los mismos, por ejemplo sincronización, clases concurrentes o clases inmutables.
- Reconocer los componentes de un sistema RPC cliente servidor.
- Implementar sistemas en RPC con serialización de parámetros y callbacks asíncronos.
- Explicar las diferentes características de storage distribuidos como particionamiento, consistencia, redundancia, etc.
- Conocer las diferentes características de las bases de datos distribuidas de tipo "NoSQL".
- Detallar diferentes estrategias y herramientas para el procesamiento distribuido.
- Implementar aplicaciones de procesamiento de información distribuidas en Map Reduce.
- Utilizar la herramienta de storage y procesamiento distribuido.
- Analizar el comportamiento de aplicaciones distribuidas al modificar la composición del cluster

## ➤ Contenidos:

| Título       | Descripción                                                                                                                                                                                                                                                                                                                  |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java +       | Java estilo Funcional, Lambdas, Tests.<br>Estructuras y features en las últimas versiones de Java                                                                                                                                                                                                                            |
| Concurrencia | Procesadores multiproceso y Programas multithread.<br>Programación de Threads en Java, Executors y pools<br>Problemas de la programación concurrente: Interferencia, Coordinación y Disponibilidad.<br>Sincronización nativa<br>Clases concurrentes de alto nivel<br>Alternativas a la sincronización<br>Completable Futures |
| RPC          | Conceptos de Cliente Servidor<br>RPC como modo de comunicación.<br>Middlewares.<br>Definición de servicios.<br>Implementación de cliente.<br>Semánticas<br>Garbage Collection Distribuido<br>Implementación de servidor.<br>Serialización de Parámetros.<br>Servicios asíncronos.                                            |

|                           |                                                                                                                                                                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APIS Alternativas         | Revisión de otras estrategia de definición de APIs alternativas a RPC                                                                                                                                                                                            |
| Sistemas Distribuidos     | <p>Conceptos de Sistemas distribuidos.</p> <p>Distribución de responsabilidades, procesamiento y guardado.</p> <p>Particionado, Distribución y Replicación de datos.</p> <p>Modelos de Consistencia.</p> <p>Coordinación, Elasticidad y Tolerancia a fallos.</p> |
| Storage Distribuido       | <p>File System Distribuidos y Arquitectura de HDFS</p> <p>Uso de Hazelcast como herramienta para guardado distribuido.</p> <p>Conceptos de NoSQL, análisis de tipo de storage, teorema C.A.P. y diversas arquitecturas de estas herramientas.</p>                |
| Procesamiento Distribuido | <p>Map Reduce como ejemplo de procesamiento distribuidos.</p> <p>Análisis de la arquitectura y comportamiento de las herramientas de procesamiento distribuidos</p>                                                                                              |

## ➤ Estrategias de enseñanza:

Clases expositivas donde se repasan los temas teórico o las herramientas y formas de codificación. Para el caso de herramientas o código se van mezclando momentos de teoría con ejercicios que los alumnos realizan para fijar los conceptos y disparar discusiones.

Como parte de la evaluación realizan dos trabajos especiales grupales y se dispone por cada trabajo:

- una clase de consulta/prueba para que puedan realizar pruebas en conjunto y consultar a la cátedra dudas del trabajo.
- una clase de coloquio, donde por grupo se revisa el trabajo, se corren pruebas, se hacen preguntas a modo de evaluación y se charlan soluciones.

## ➤ Actividades:

| Título                           | Descripción                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clases teórico prácticas         | Clases semanales expositivas donde se abordan los conceptos teóricos, se muestra código de ejemplo y/o configuraciones de las herramientas.<br>En los temas donde se programa y/o utilizan herramientas, se intercalan pequeños ejercicios prácticos para que los alumnos fijen el tema, puedan probar funcionalidad o se encuentre con situaciones que funcionan como disparador de lo que se discutirá a continuación |
| Prácticas                        | Para algunos temas se habilitan prácticas con ejercicios para desarrollar programas y que los alumnos puedan ampliar su ejercitación, probar funcionamientos y generar dudas.                                                                                                                                                                                                                                           |
| Consultas vía debates de campus. | Más allá del horario de clases los alumnos cuentan con la posibilidad de realizar consultas vía campus ya sea para las prácticas como para los trabajos especiales                                                                                                                                                                                                                                                      |
| Trabajos especiales              | La materia cuenta con dos trabajos especiales integradores como evaluación.<br>Estos trabajos son grupales y en los mismos los alumnos se encuentran con un problema más "real" y "complejo" que en las prácticas y desarrollan una solución aplicando los conceptos vistos en clase.<br>La mayor parte de estos trabajos se realiza fuera del horario de clase.                                                        |
| Clase de consulta y prueba       | Para cada TPE se dispone de una clase donde los alumnos pueden realizar consultas y más que nada realizar pruebas de integración en los equipos                                                                                                                                                                                                                                                                         |
| Coloquio                         | Para cada trabajo práctico se realiza un coloquio donde se corren pruebas, se revisa la solución propuesta haciendo preguntas a los integrantes del equipo y se plantean alternativas de diseño                                                                                                                                                                                                                         |

## ➤ Recursos didácticos:

---

- Campus para la distribución de material. Salas virtuales por si la clase es remota.
  - Presentación vía Google Slides para la teoría
  - Documentos de Google docs para las prácticas
  - Proyector para la presentación presencial.
  - Computadora por alumno para las prácticas.
  - Conexión a pampetro
- 
- Programas a instalar: Java, maven, docker y docker compose (versiones a determinar por cuatrimestre)

## ➤ Modalidad de evaluación y aprobación:

---

### Modalidad de evaluación:

La cursada se evalúa mediante a dos trabajos prácticos especiales grupales.

Para estos trabajos prácticos la cátedra provee una lista de requerimientos y pautas a cumplir por un sistema y los alumnos los desarrollan fuera del horario de clase aplicando los conceptos vistos en clase.

En caso de que alguno de los trabajos prácticos no sea aprobado se recupera mediante a la corrección del mismo, agregado de más funcionalidad o preguntas teórico/prácticas.

La evaluación de la materia se completa con un final teórico/práctico individual presencial con preguntas de los temas vistos en clase.

Las preguntas del final pueden ser:

- definiciones teóricas
- preguntas teóricas a desarrollar
- análisis de casos prácticos,
- análisis de código
- desarrollo de código.

### Requisitos de aprobación:

Para aprobar la materia se deben tener aprobados ambos trabajos prácticos (ya sea directamente o a través de sus recuperatorios).

La nota de cursada será el promedio de la nota de ambos.

$$\text{NotaCursada} = 0.5 * \text{Nota1erTPE} + 0.5 * \text{Nota2doTPE}$$

La nota de cada trabajo es la nota del trabajo aprobado en primera instancia, o el promedio entre la nota del trabajo desaprobado y su recuperatorio aprobado. Si este promedio da menor a 4 la nota queda en 4.

El final se aprueba con 4 o más.

## ➤ Bibliografía obligatoria:

---

| N° | Descripción                                          |
|----|------------------------------------------------------|
| 1  | Presentaciones y documentos provistos por la cátedra |

➤ **Bibliografía complementaria:**

---

| N° | Descripción |
|----|-------------|
| 1  |             |